01-lab-work-submission

S3: Lab Work

Revix · *driven by reviews.* What we submit, where each mark lives, and how to walk through it.

Submission	S3 Lab work
Weight	**40 marks** of 100
Due	**Friday 2 October 2026**
Team	Aditya Nariyapara, Devika Jonjale, Saachi Shinde
Repository	`saaachis/revix-evidence-traceable-review-system`
Live API	`https://revix-api-tcyq.onrender.com`
Live site	`https://revix-reviews.vercel.app`
Comes after	Review 1: concept, market gap and literature review

> **Figures in this document were taken on 7 September 2026.** They grow with every nightly run. Refresh them before submitting: section 7 gives the two commands that produce every number quoted here.

---

1. The four things being marked

The rubric splits 40 marks four ways. This document is organised in exactly that order, so each section can be opened against the mark it answers.

#	Marked on	Marks	Section	The one-line answer
1	Codebase	10	§3	Three Python packages and a Next.js app, ~15,200 lines, running on real data
2	Frameworks with justification	10	§4	Every choice is written down as a decision record, including what we rejected
3	Code quality	10	§5	Strict types, 233 tests, six CI jobs, and defects our own tests caught
4	NFRs achieved	10	§6	All nine categories audited; six met by design, five gaps found and closed

2. What Revix is, in sixty seconds

Star ratings hide more than they show. A 4.2 average tells you nothing about *who* rated, *what* they cared about, or *whether they agreed*. Revix rebuilds the verdict from the reviews underneath it and keeps every number traceable to the sentences it came from.

Three claims, and the screen has to prove each one:

Claim	What proves it on screen
We weight by trust, not by star average	A switch that changes the numbers when flipped
We are honest about uncertainty	A confidence range, not a single decimal
Every number is traceable	Every score opens the reviews behind it

Where the data comes from. Owner reviews from CarWale and BikeWale, CarDekho and BikeDekho, and comments on YouTube review videos. All of it real, all of it collected under the sources' own rules.

Current scale.

Evidence units	**15,853**
Variants in the catalogue	**143** across 42 models and 16 manufacturers
Variants with a published verdict	**134** (9 held back below the evidence floor)
Two-wheelers	47 in the catalogue, 42 published
Verdicts stored	429 (143 variants × 3 weighting schemes)
Evidence attributed to verdicts	14,137 units, averaging 99 per variant

3. Codebase (10 marks)

3.1 Shape

A uv workspace holding three Python packages and one Next.js application.

```
revix/
├── packages/revix_core/   models, settings, session          1,294 lines
├── pipeline/              ingest, extract, resolve, fuse      6,716 lines
├── apps/api/              the read-only serving layer           857 lines
├── apps/web/              Next.js 16 frontend                 3,529 lines
├── tests/                 15 modules, 233 tests               2,808 lines
└── docs/                  ADRs, proposal, this document
```

Roughly **15,200 lines** across **56 Python files** and **21 TypeScript files**, not counting generated code or lockfiles.

3.2 The one rule that holds it together

Dependencies run in one direction only:

```
  revix-api ──────┐
                  ├──> revix-core   (models, settings, session)
  revix-pipeline ─┘
```

`revix_core` never imports from the other two. That is what stops the read path and the write path tangling into each other, and it is **enforced by import linting in CI**, not left to discipline. Worth pointing at directly: architecture that is only a diagram decays, and architecture a build fails on does not.

3.3 The database mirrors the same idea

Four Postgres schemas, separated by lifecycle rather than by subject:

Schema	Holds	Written by
`raw`	Fetched payloads, content-hashed	Ingest
`core`	Catalogue, sources, evidence units	Ingest and resolution
`analysis`	Aspect scores, evaluation runs	Enrichment and fusion
`serving`	Finished verdicts, claims, citations	Fusion, read by the API

You can always tell which layer a table belongs to and what is allowed to write to it. Schema changes go through Alembic migrations, and CI checks they apply, that they match the models, and that they **reverse cleanly**.

3.4 It runs on real data, unattended

A scheduled GitHub Actions workflow runs the whole pipeline nightly: fetch, extract, resolve, fuse, publish. The most recent full run took **53m 28s** inside a 120-minute timeout. Nobody starts it and nobody watches it.

3.5 What to open, in order

1. `pyproject.toml`, the workspace and the dependency rule, with the diagram in a comment
2. `packages/revix_core/src/revix_core/models/`, the typed ORM, four schemas
3. `pipeline/src/revix_pipeline/`, connectors, enrichment, fusion
4. `apps/api/src/revix_api/main.py`, every endpoint, each one an indexed read
5. `apps/web/src/app/`, ten routes, all server-rendered

---

4. Frameworks with justification (10 marks)

The artefact for this mark is the ADR folder. Eight architecture decision records in `docs/adr/`, each stating the decision, the alternatives, and the consequence we accepted. This section summarises them; open the ADR itself for any the examiner pushes on.

4.1 Backend

Framework	Why this one	What we did not do
FastAPI	Contract-first. The OpenAPI schema is the source of truth and the frontend's TypeScript client is generated from it, so the two *cannot* drift	Flask, which would have meant hand-writing and hand-syncing the client
Pydantic v2	Validation at the boundary, so a malformed request never reaches a query	Manual validation scattered through handlers
SQLAlchemy 2.0 typed ORM	Bound parameters everywhere, so injection has no surface, and the models type-check	Raw SQL, faster to write and impossible to refactor safely
Alembic	Reversible migrations, verified in CI	Hand-applied schema changes
PostgreSQL 16 with `pgvector` and `pg_trgm`	One datastore does relational, vector similarity *and* fuzzy text. Variant resolution needs trigram matching; near-duplicate detection needs vectors	A separate vector database, which is a second service to deploy and keep consistent for a workload this size
Typer	The pipeline is a CLI, so every stage is runnable and debuggable on its own	A monolithic script

4.2 Frontend

Framework	Why this one
Next.js 16, App Router	Server rendering, so a verdict page is a real URL that can be shared and indexed. Structured data matters for a review product
React 19	Component model, and what Next targets
Tailwind v4	CSS-first `@theme`, so the palette is one place. It had to be reworked once for contrast, and one place is why that was cheap
openapi-typescript	Generates the client from the live schema. See §5.2

4.3 The two choices worth defending out loud

GitHub Actions instead of Airflow or Prefect (ADR 0002). An orchestrator is a server to run, secure and pay for. Our pipeline is one nightly batch with no fan-out and no backfill. Actions already holds our secrets, already runs on a schedule, and already gives logs and alerting. Choosing the heavier tool would have been choosing a tool because it looks professional rather than because the problem needs it.

A uv workspace with three packages instead of one flat project (ADR 0003). Three packages make the dependency rule in §3.2 real: the API physically cannot import the pipeline. One package would have made that a naming convention, and naming conventions lose.

4.4 Machine learning, and the honest result

We built a distant-supervision aspect classifier: lexicon labels train a TF-IDF into a one-vs-rest logistic regression. Held against the lexicon it learned from, **it lost by 0.29 macro F1, so it ships disabled**.

That is the measurement working, not the project failing. The alternative was shipping a model because we had built one. ADR 0004 records why the baseline came first and why it stayed.

---

5. Code quality (10 marks)

5.1 Enforced, not encouraged

Control	Setting
Type checking	**mypy strict** across all three Python packages; TypeScript strict
Linting	**ruff** check and format, both failing the build
Tests	**233 tests** in 15 modules; 215 run without a database, the rest are database-marked and run in CI
Browser	Playwright end-to-end smoke test
Accessibility	**axe-core against all nine pages**, WCAG 2.1 AA, failing the build on a violation

Six CI jobs run on every pull request. Beyond the usual, they check that **migrations reverse cleanly**, that the **pipeline runs end to end**, that **no secret and no raw scraped payload is ever committed**, and that **every internal documentation link resolves**.

5.2 The contract cannot drift

CI regenerates the TypeScript client from the live OpenAPI schema and **fails if the committed types differ**. A response shape that changes without the frontend changing is a red build, not a runtime surprise a user finds. This is the single strongest quality guarantee in the project because it is structural: it does not depend on anybody remembering.

5.3 Defects our own tests caught

The most useful evidence that the tests are real is what they found. Offer these if asked how we know the suite is worth anything:

Defect	Why it mattered
Macro-F1 averaged over zero-support aspects	A perfect system scored 0.22 on a two-topic set. Our evaluation was lying to us in our favour
A connector recorded our own search term as the source's answer	We would have been citing ourselves as evidence
An unordered `LIMIT` in fusion	Ingest and fusion silently disagreed about which variant they meant
Security headers added inside the rate limiter	A 429, the response most likely to reach a hostile client, was the one response going out bare
A migration emitting `NOT NULL` with no default	Would have failed against the 129 verdict rows already in production

5.4 Comments explain the why

House style is that a comment explains a decision, never restates the code. A representative example, on why `/health` opens its own session:

> *Deliberately not using the session dependency. If it did, an unreachable database would raise during dependency resolution, before the handler ran, and the health endpoint would answer 500 with a SQLAlchemy stack trace: no diagnosis for us and a stack trace for everyone else.*

That is the kind of comment that survives a rewrite, because it carries information the code cannot.

---

6. NFRs achieved (10 marks)

The artefact for this mark is `docs/non-functional-requirements.md`. It works through all nine standard categories against the actual code rather than against what our proposal claimed. Open it directly; this is a summary.

6.1 All nine, and where each is met

#	Requirement	State	Where it is met
1	**Performance**	Met, measured	No model runs on the read path; every endpoint is one indexed read of a precomputed row
2	**Scalability**	Met for the stated load	Pagination capped at 200, bounded pool, expensive work batched overnight
3	**Portability**	Met	Docker, uv lockfile, configuration only through environment variables
4	**Usability**	Met, audited	WCAG 2.1 AA clean on all nine pages, enforced in CI
5	**Compatibility**	Met	Generated API client, responsive layout, standards only
6	**Security**	Met at this threat model	Headers, CORS allow-list, rate limiting, read-only API, no secrets in the repo
7	**Reliability**	Met	Sources degrade independently, every wait bounded, no traceback ever leaves
8	**Maintainability**	Met	Strict types, 233 tests, ADRs, generated client
9	**Availability**	Partly met, stated honestly	503 health contract and keep-warm; a single free-tier instance is the known limit

6.2 The point worth making first

Six of the nine were already satisfied, and **almost all of them by a single architectural decision**: the strict separation of the write path from the read path. No model runs during a user request. Every endpoint reads a row the nightly pipeline already computed.

That one decision simultaneously delivers performance (nothing on the read path grows with the corpus), scalability (reads do not scale with data at all), reliability (a page cannot fail because a third-party model is down) and availability (a demo depends on nothing remote).

Say this before the list. It is the difference between "we implemented nine things" and "we made one decision that satisfied six requirements, and then closed the rest deliberately."

6.3 What the audit found and fixed

Five real gaps, found by reading the code rather than the proposal:

1. **No security headers anywhere.** Neither Vercel nor FastAPI adds any by default, so the site shipped sniffable, framable, and sending a full referrer to every outbound link, and every outbound link here points at a review on somebody else's site. Both now send CSP, `nosniff`, `DENY`, `strict-origin-when-cross-origin`, `Permissions-Policy` and HSTS.

2. **Unbounded database waits, which defeated a contract we had already written and tested.** This is the one to lead with; see §6.4.
3. **No rate limiting.** One scraper walking every variant id could exhaust the connection pool every other reader was queued behind. Now a sliding window, 120 requests per minute per client, with `Retry-After` and the remaining budget published so a well-behaved client can slow down *before* it is refused.
4. **No compression or caching.** Every client re-fetched the full catalogue uncompressed. Measured at **55.3 kB down to 5.9 kB gzipped, 9.3× smaller**, plus cache directives that are only safe because of the write/read split.
5. **No request identity.** A failure report could not be tied to a request. Every response now carries `X-Request-ID` and `X-Response-Time-ms`, which also makes our p95-under-300ms claim checkable from outside the process instead of only by a benchmark we ran on ourselves.

6.4 The finding to lead with

`/health` was deliberately built to report an unreachable database as a 503 rather than a 500 with a stack trace. It was written that way on purpose, and it was tested.

It could not actually do it. With no connect timeout, psycopg waited on the operating system's TCP timeout while the platform's health probe gave up first. A correct 503 became no answer at all, and a hang is a worse failure than an error because nothing downstream can react to it.

What led us there was our own test run hanging for four hundred seconds against a machine with no Postgres. Every database wait is now bounded: connect 5s, pool wait 10s, recycle 240s ahead of Neon dropping idle connections.

This is worth telling as a story because it shows the difference between *writing* a non-functional requirement and *verifying* one. We had the code, the intent and the test, and the requirement still was not met.

6.5 Measured, not asserted

Metric	Value
API response, warm, local	22–35 ms
API response from India, deployed	96–247 ms (target: p95 under 300 ms)
Compare page render, warm	0.35 s
Catalogue payload	55.3 kB → 5.9 kB gzipped
Accessibility	0 violations, WCAG 2.1 AA, nine pages
API container image	364 MB, non-root uid 10001
Nightly pipeline	53m 28s inside a 120-minute timeout

13 tests in `tests/test_nfr.py` assert the headers, the limiter, the error shape and the compression. These are exactly the things that work the day they are added and quietly stop working six commits later, because nothing looks broken

when they are gone.

6.6 Two limits we state rather than hide

Both are in the audit document in writing:

- The rate limiter is **per instance, in memory**. It is not a global quota. Two instances would each allow the full rate.
- A single free-tier instance is **resilient to its dependencies failing, not highly available**. There is no redundancy or failover.

Both are the right call at this size, and both are the first thing that would need replacing if Revix grew. Stating the limit before being asked reads as engineering judgement; being caught at it does not.

---

7. Refreshing every number in this document

```
# Codebase and tests
find packages pipeline apps/api tests -name "*.py" -not -path "*/__pycache__/*" | wc -l
uv run pytest -q                        # test count
uv run ruff check . && uv run mypy packages/revix_core/src pipeline/src apps/api/src

# Live scale, straight from the deployed API
curl -s https://revix-api-tcyq.onrender.com/health
curl -s https://revix-api-tcyq.onrender.com/sources/health

# The NFR guarantees
uv run pytest tests/test_nfr.py -q
cd apps/web && npm run a11y
```

---

8. Handing it in

#	Item	Where
1	The repository, `main` branch, green CI	GitHub
2	This document	`docs/s3-lab-work/`
3	The NFR audit	`docs/non-functional-requirements.md`
4	Eight architecture decision records	`docs/adr/`
5	Live API and live site	Links in the header

Before submitting: confirm `main` is green, re-run the commands in §7 and update any figure that has moved, and check the live site loads from a phone as well as a laptop.